

浙江大学实验报告

课程名称: 数字系统设计实验
实验名称: 实验 19 音乐播放实验

指导老师: 屈民军, 唐奕
实验类型: 设计实验

一、实验目的

- (1)掌握音符产生的方法, 了解 DDS 技术的应用。
- (2)了解音频解码的应用。
- (3)掌握系统“自顶而下”的数字系统设计方法。

二、实验任务

设计一个音乐播放器, 要求以下条件。

- (1)可以播放四首乐曲, 设置 play/pause button、next button、reset 三个按键。按 play/pause button 键, 音乐在播放和暂停之间切换;按 next button 键播放下一首乐曲。
- (2)LED0 指示播放情况(播放时点亮)、LED2 和 LED3 指示当前乐曲序号。

三、实验原理

1、DDS 原理

要在数字域产生正弦信号, 可以用一个存储器 (ROM/RAM) 存储一张正弦表; 然后将存于表中的正弦样品取出, 经数模转换器 D/A, 形成模拟量波形。DDS 技术通过改变寻址的步长来改变输出信号的频率, 步长为对数字波形查表的相位增量, 输出正弦频率与相位增量成线性关系。

DDS 的基本原理框图如图 1 所示, 由相位累加器、正弦查询表 (Sine ROM)、D/A 转换器和低通滤波器组成。

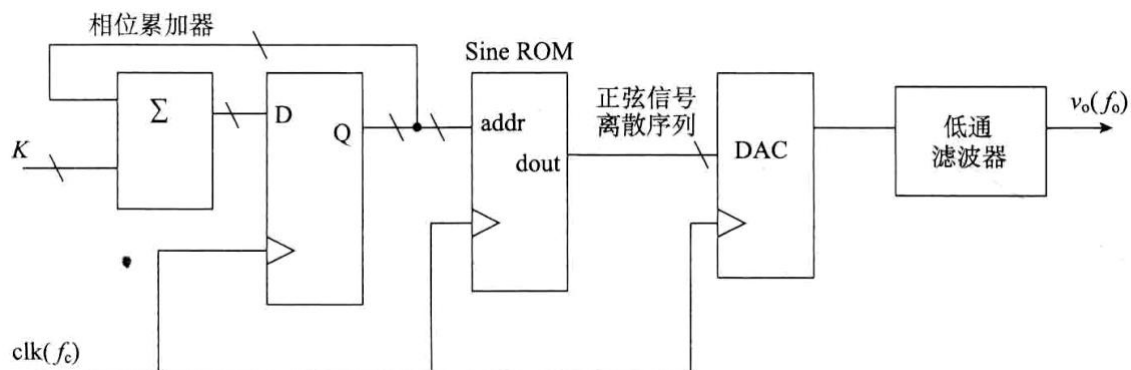


图 1 DDS 原理框图

Sine ROM 中存放一个完整的正弦信号样品, 正弦信号样品根据如下的映射关系构成, 即

$$S(i) = (2^{n-1} - 1) \times \sin\left(\frac{2\pi i}{2^m}\right), \quad i = 0, 1, 2, \dots, 2^{m-1}$$

式中, m 为 Sine ROM 地址线位数; n 为 ROM 数据线宽度; $S(i)$ 的数据形式为补码。

f_c 为取样时钟 clk 的频率, K 为相位增量 (也称频率控制字), 输出正弦信号的频率 f_o 由 f_c 和 K 共同决定, 即

$$f_o = \frac{K \times f_c}{2^m}$$

由上式可以看出，正弦信号的频率 f_0 与相位增量 K 呈正比关系。相位累加器的位数由 m 位整数和 p 位小数组成，其高 m 位整数部分作为 Sine ROM 的地址。

因为 DDS 遵循奈奎斯特(Nyquist)取样定律，即最高的输出频率是时钟频率的一半，即 $f_0 \leq f_c/2$ 。实际中 DDS 的最高输出频率由允许输出的杂散电平决定，一般取值为 $f_0 \leq 40\% \times f_c$ ，因此 K 的最大值一般为 $40\% \times 2^{m-1}$ 。

DDS 可以很容易实现正弦信号和余弦信号正交两路输出，只需用相位累加器的输出同时驱动固化有正弦信号波形的 Sine ROM 和余弦信号波形的 Cos ROM，并各自经数模转换器和低通滤波器输出即可。

另外，DDS 也容易实现调幅和调频，图 2 和图 3 所示为 DDS 实现调幅和调频的原理框图。

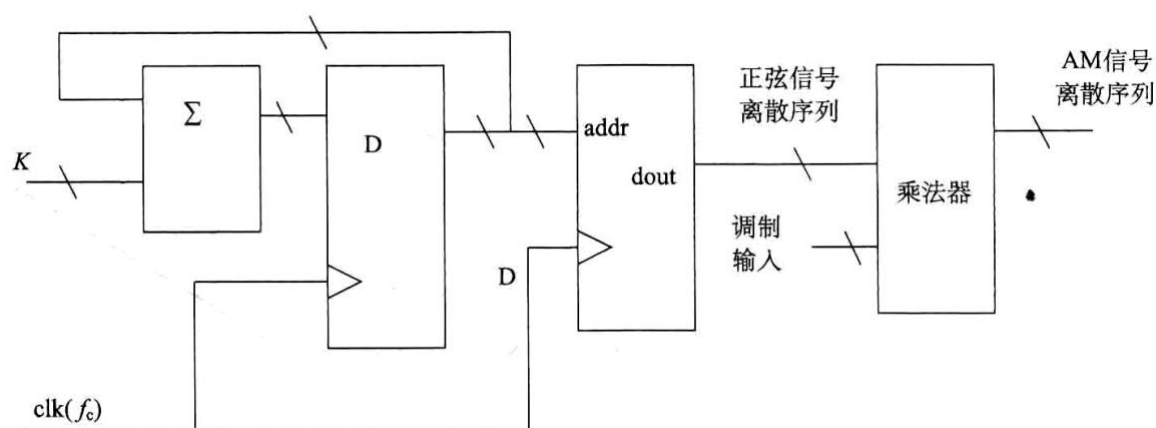


图 2 DDS 实现调幅原理框图

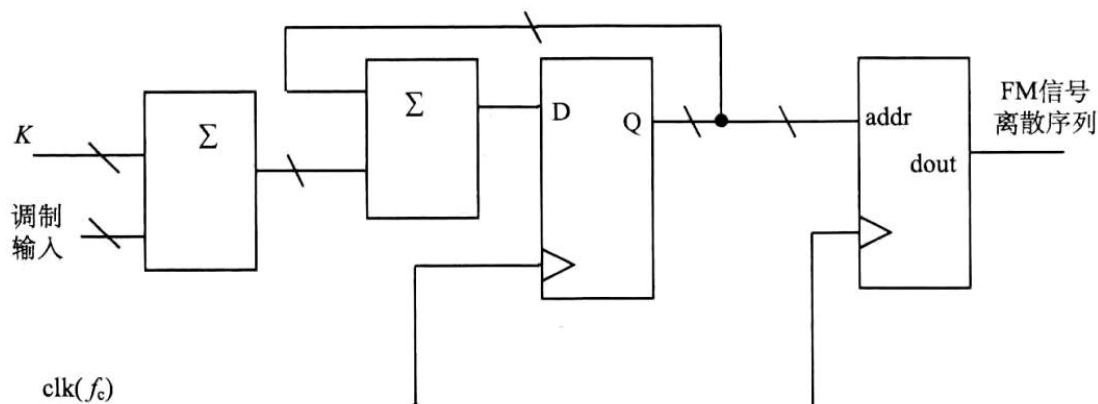


图 3 DDS 实现调频原理框图

根据 DDS 输出信号的最低频率要求，可计算出 $m=12$ 。但为了得到更准确的正弦信号频率，相位累加器位数会增加 10 位小数。所以相位累加器为 22 位累加器，高 12 位为 Sine ROM 的地址。

因为 $m=12$ ，所以存储一个完整周期的正弦信号样品就需要 $2^{12} \times 16\text{bits}$ 的 ROM。但由于正弦波形的对称性，将正弦波形分为四个区域，只需要在 Sine ROM 中存储 1/4 的正弦信号样品即可。这样，Sine ROM 容量可减少为 $2^{10} \times 16\text{bits}$ ，即 10 位地址，存储 1/4 的正弦信号样品，共 1024 个。

上述有一点需注意，1/4 周期的正弦信号样品未给出 90° 的样品值，因此在 ROM 地址为 1024 (即 90°)时可取地址为 1023 的值 (实际上地址为 1023 时，正弦信号样品已达最大值)。

因为必须利用 1/4 周期的 1024 个样品复制出一个完整正弦周期的 4096 个样品，所以对 DDS 结构要进行必要处理，优化后的 DDS 结构如图 4 所示。

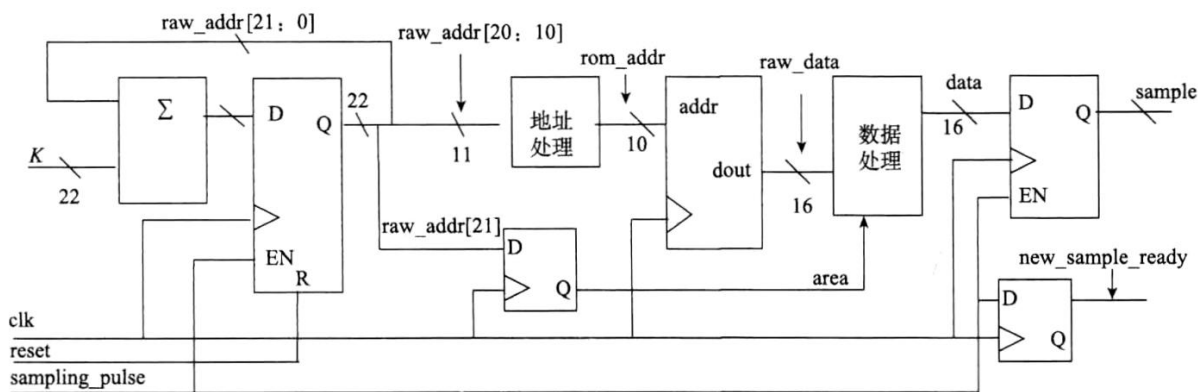


图4 优化后的 DDS 结构

2、音乐播放器的顶层设计

根据实验任务可将系统划分为时钟管理模块(DCM)、按键处理、主控制器、乐曲读取、音符播放(note_player)、同步化电路、节拍基准产生器和音频编解码接口电路等子模块,如图5所示。各主要子模块作用如下。

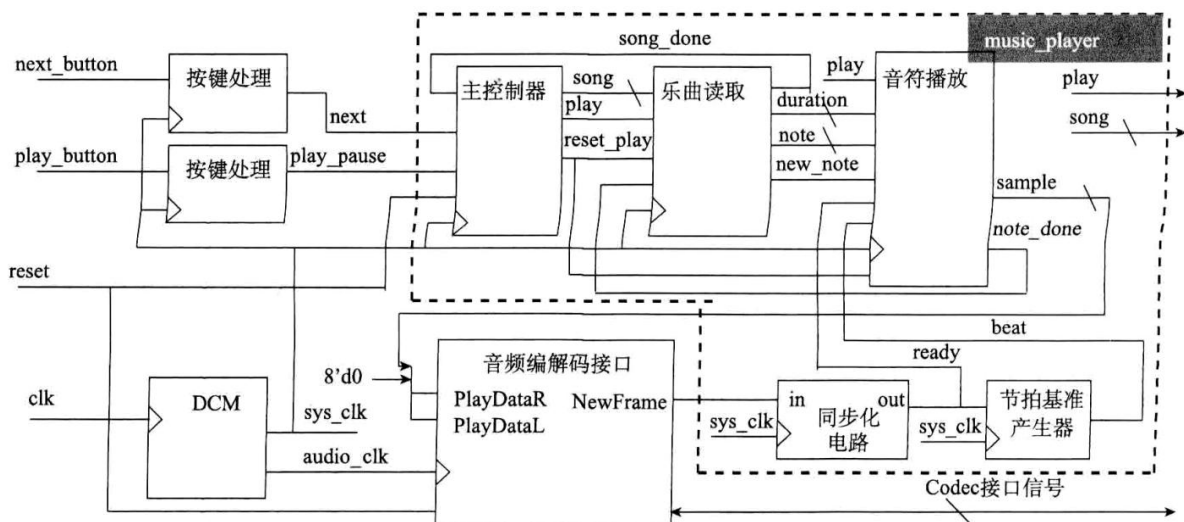


图5 音乐播放器的顶层框图

时钟管理模块(DCM)产生 100MHz 的系统时钟 sys_clk 和 12.5MHz 的音频时钟 audio_clk。

主控制器(mcu)模块接收按键信息,通知 song_reader 模块是否要播放(play)及播放哪首乐曲(song)。

乐曲读取(song_reader)模块根据 mcu 模块的要求,逐个取出音符信息 {note,duration} 送给 note_player 模块播放,当一首乐曲播放完毕,回复 mcu 模块乐曲播放结束信号(song_done)。

音符播放接收到需播放的音符,在音符的持续时间内,以 48kHz 速率送出该音符的正弦波样品给音频编解码接口模块。当一个音符播放结束,向 song_reader 模块发送一个 note_done 脉冲索取新的音符。

音频编解码接口模块负责将音符的正弦波样品转换为串行输出并发送给音频编解码芯片 ADAU1761。音频编解码芯片 ADAU1761 接收正弦波样品,再进行 AD 转换并放大,最后送至扬声器播放。注意,note_player 模块产生的正弦波样品为 16 位二进制,需在低位加 8 个 0 后送入音频编解码接口模块。

由于音频编解码模块与系统使用不同时钟,因此需要同步化电路协调两部分电路。

节拍基准产生器产生 48Hz 的节拍定时基准脉冲信号(beat),而 ready 信号频率为 48kHz,因此,节拍基准产生器为分频比为 1000 的分频器。而按键处理模块完成输入同步化、防颤动和脉宽变换等功能。

3、主控制模块 mcu 的设计

主控制模块 mcu 有响应按键信息、控制系统播放两大任务，表 1 为其端口含义。

引脚名称	I/O	引脚说明
clk	Input	100MHz 时钟信号
reset	Input	复位信号，高电平有效
play_pause	Input	来自按键处理模块的“播放/暂停”控制信号，一个时钟周期宽度的脉冲
next	Input	来自按键处理模块的“下一曲”控制信号，一个时钟周期宽度的脉冲
play	Output	输出控制信号，高电平表示播放，控制 song_reader 模块是否要播放
reset_play	Output	时钟周期宽度的高电平复位脉冲 reset_play，用于同时复位模块 song_reader 和 note_player
song_done	Input	song_reader 模块的应答信号，一个时钟周期宽度的高电平脉冲，表示一曲播放结束
song[1:0]	Output	当前播放乐曲的序号

表 1 主控制模块 mcu 的端口含义

根据设计要求，模块 mcu 的原理框图如图 6 所示。图中的 2 位二进制计数器用来计算乐曲序号 (song)。

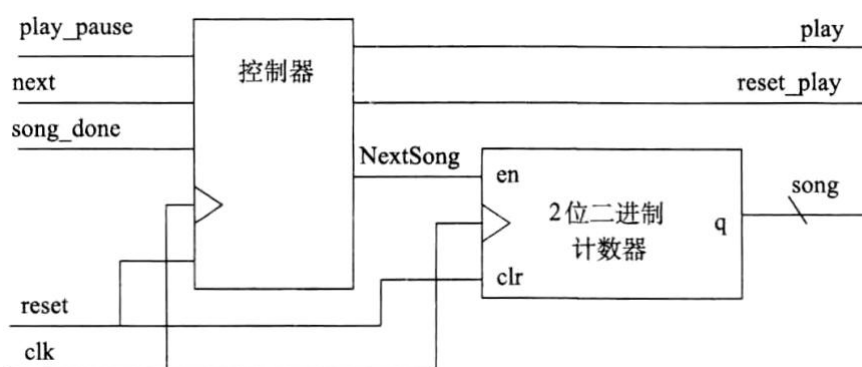


图 6 mcu 的结构框图

控制器的工作流程图如图 7 所示，控制器设置初始复位 (RESET)、播放 (PLAY)、暂停 (PAUSE) 和下一首 (NEXT) 四种状态。系统复位后，经 RESET 状态初始化后进入 PAUSE 状态，等待各种命令输入；play_pause 脉冲信号使系统在 PLAY、PAUSE 两状态之间互转；在 PLAY 或 PAUSE 状态下，若按下 next_button 按钮，则使系统在进入 NEXT 状态，输出 reset_play 脉冲复位 song_reader 和 note player 两个模块，同时输出脉冲 NextSong 乐曲序号计数器加 1，进入下一曲播放；另外，在 PLAY 状态时，若乐曲播放结束 (song_done 有效) 则结束播放，经 RESET 状态复位 song_reader 和 note player 两个模块，并进入 PAUSE 状态，再次等待各种命令输入。

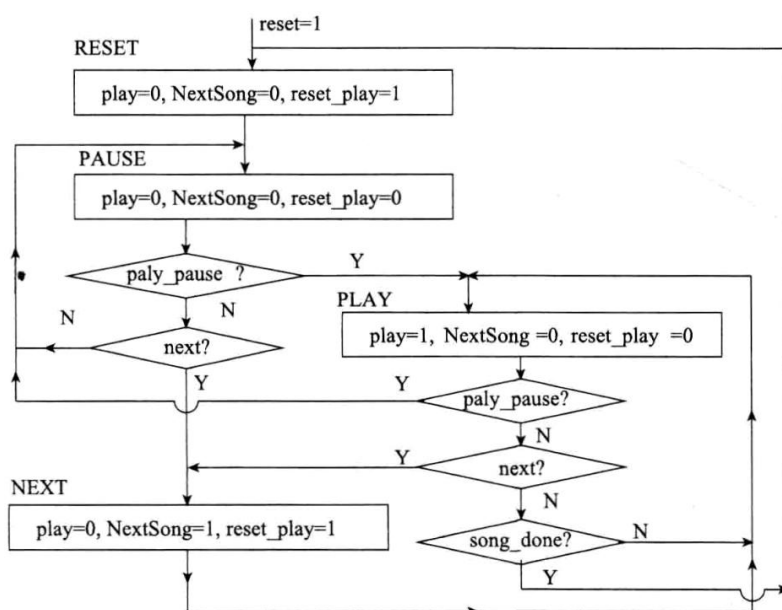


图 7 mcu 控制器的算法流程图

4、乐曲读取模块 song_reader 的设计

乐曲读取模块 song_reader 的任务有

- (1)根据 mcu 模块的要求，选择播放乐曲。
- (2)响应 note_player 模块请求，从 song_rom 中逐个取出音符{note,duration}送给 note player 模块播放。
- (3)判断乐曲是否播放完毕，若播放完毕，则回复 mcu 模块应答信号。

根据 song_reader 模块的任务要求，song_reader 模块需包含表 2 所示的输入、输出端口。

引脚名称	I/O	引脚说明
clk	Input	100MHz 时钟信号
reset	Input	复位信号，高电平有效
play	Input	来自 mcu 的控制信号，高电平要求播放
song[1:0]	Input	来自 mcu 的控制信号，当前播放乐曲的序号
note_done	Input	即模块 note_player 的应答信号，一个时钟周期宽度的脉冲，表示一个音符播放结束并索取新音符
song_done	Output	给 mcu 的应答信号，当乐曲播放结束，输出一个时钟周期宽度的脉冲，表示乐曲播放结束
note[5:0]	Output	音符标记
duration[5:0]	Output	音符的持续时间
new_note	Output	给模块 note_playe 的控制信号，一个时钟周期宽度的高电平脉冲，表示新的音符需播放

表 2 乐曲读取模块 song_reader 的端口含义

song_rom 是一个只读存储器，用来存放乐曲，容量为 $2^7 \times 12\text{bits}$ 。共存放四首乐曲，每首乐曲占用 $2^5 \times 12\text{bits}$ 空间，即每首乐曲最长由 32 个音符组成。因此，song_rom 高 2 位地址决定哪首乐曲，而低 5 位地址决定这首乐曲的哪个音符。song_rom 每个地址存放一个音符信息，音符信息由 12 位二进制组成，高 6 位表示音符标记 note，低 6 位表示音长 duration。

根据 song_reader 模块的功能及 song_rom 结构，可画出图 8 所示的结构框图，控制器主要负责接收 mcu 模块与 note_player 模块的控制信号，并做出响应。算法流程图如图 9 所示。

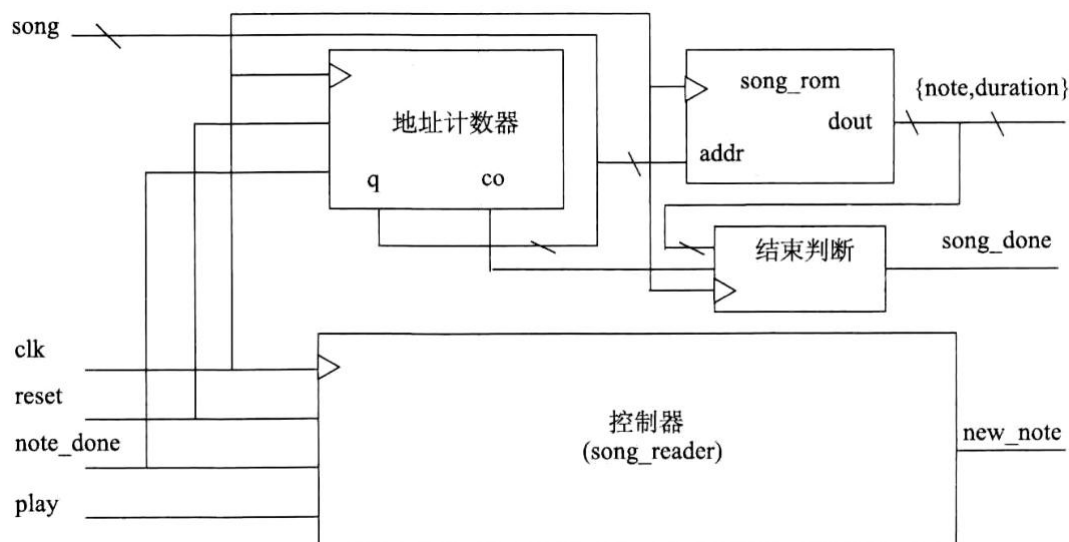


图 8 song_reader 的结构框图

系统复位后一直在 RESET 状态等待 mcu 模块控制信号输入，当 mcu 模块发出播放命令(play 为高电平)时，进入 NEW_NOTE 状态输出 new_note 脉冲要求 note_player 模块播放音符；然后进入 WAIT 状态等待，当 note_player 模块播放完音符时，会发出 note_done 脉冲信号索取下一音符，note_done 脉冲信号一方面让地址计数器递增，并从 song_rom 取出一个新的音符，另一方面让控制

器进入 NEW_NOTE 状态，输出 new_note 脉冲通知 note_player 模块有新的音符需要播放。当 note_done 有效，需要两个时钟周期才能从 song_rom 中读取下一个音符信息。因此新音符有效标记信号 new_note 也应在新音符数据输出后有效。所以在流程图中插入 NEXT_NOTE 状态，目的是延迟一个时钟周期输出信号，以配合 song_rom 的读取要求。

地址计数器为 5 位二进制计数器，其中 note_done 为计数使能输入，当 note_done 为高电平时，允许计数。计数器状态 q 为 song_rom 的低 5 位地址，song[1:0] 为 song_rom 高两位地址。

当地址计数器出现进位或 duration 为 0 时，表示乐曲结束，应输出一个时钟周期宽度的高电平脉冲信号 song_done。由于 duration 为 0 的持续时间可能不止 1 个时钟周期，因此“结束判断”电路模块可由状态机方法实现。

5、音符播放模块 note_player 的设计

音符播放模块 note_player 是本实验的核心模块，它主要任务包括以下几方面。

- (1) 从 song_reader 模块接收需播放的音符 {note,duration}。
- (2) 根据 note 值找出 DDS 的相位增量 k 。
- (3) 以 48kHz 速率从 Sine ROM 取出正弦样品送给音频编解码器接口模块。
- (4) 当一个音符播放完毕，向 song_reader 模块索取新的音符。

根据 note_player 模块的任务，进一步划分功能单元，如图 10 所示，图中 FreqROM 为只读存储器，完成音符标记 note 与 DDS 模块的相位增量 k 查找表关系。表 3 所示为 note_player 模块的端口含义。

引脚名称	I/O	引脚说明
clk	Input	系统时钟信号，外接 sys_clk
reset	Input	复位信号，高电平有效，外接 mcu 模块的 reset_play
play_enable	Input	来自 mcu 模块的 play 信号，高电平表示播放
note_to_load[5:0]	Input	来自 song_reader 模块的音符标记 note，表示需播放的音符
duration_to_load[5:0]	Input	来自 song_reader 模块的音符持续时间 duration，表示需播放音符的音长
load_new_note	Input	来自 song_reader 模块的 new_note 信号，一个时钟周期宽度的高电平脉冲，表示新的音符需播放
note_done	Output	给 song_reader 模块的应答信号，一个时钟周期宽度的高电平脉冲，表示音符播放完毕
sampling_pulse	Input	来自同步化电路模块的 ready 信号，频率 48kHz，一个时钟周期宽度的高电平脉冲，表示索取新的正弦样品
beat	Input	定时基准信号，频率为 48Hz 脉冲，一个时钟周期宽度的高电平脉冲
sample [15:0]	Output	正弦样品输出

表 3 note_player 模块的端口含义

note_player 控制器负责与 song_reader 模块接口，读取音符信息，并根据音符信息从 Frequency ROM 中读取相应相位增量 k 送给 DDS 子模块。另外，note_player 控制器还需要控制音符播放时间。note_player 控制器的算法流程如图 11 所示。在复位或未播放时，控制器处于 RESET 状态或 PLAY

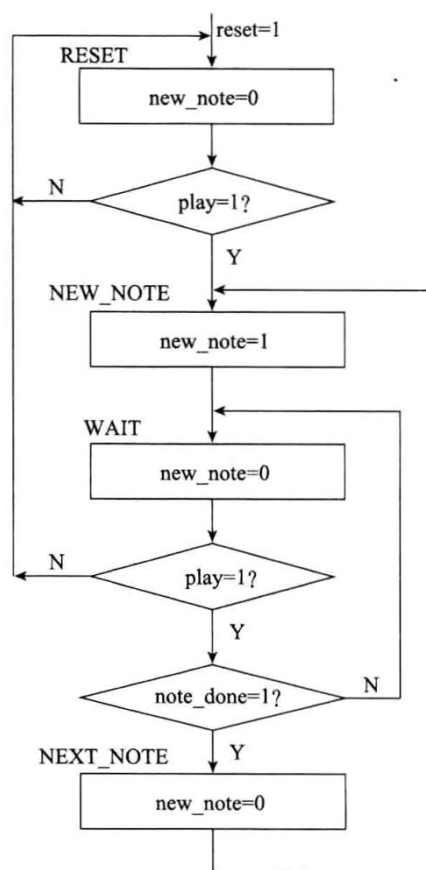


图 9 song_reader 控制器的算法流程

状态，由于此时高电平 `reset` 或低电平 `play_enable` 都使 D 型寄存器清 0，进而使 k 为 0，不会输出正弦样品。当 `play_enable` 为高电平，系统进入音符播放 PLAY 状态，当一个音符播放结束时，控制器进入 DONE 状态，置位 `done_with_note`，向 `song_reader` 模块索取新的音符，此时 `song_reader` 模块输出一个 `new_note` 脉冲信号使控制器进入 LOAD 状态，读取新的音符，然后进入 PLAY 状态播放下一个音符。

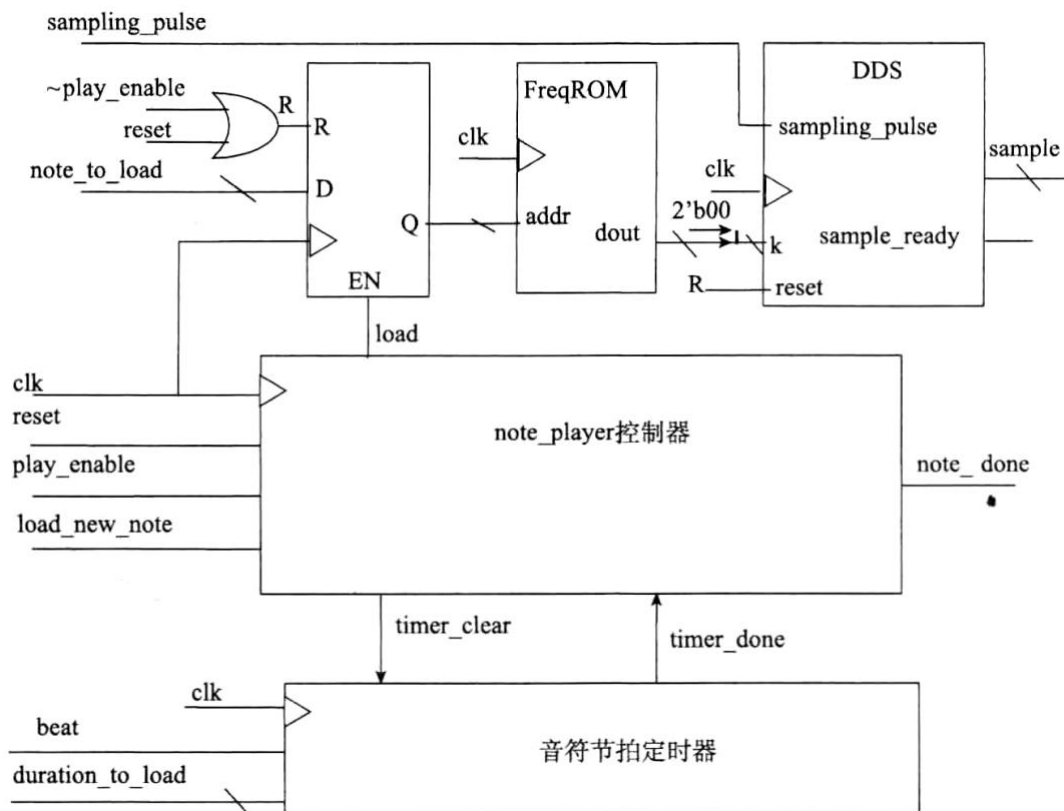


图 10 note_player 模块的结构框图

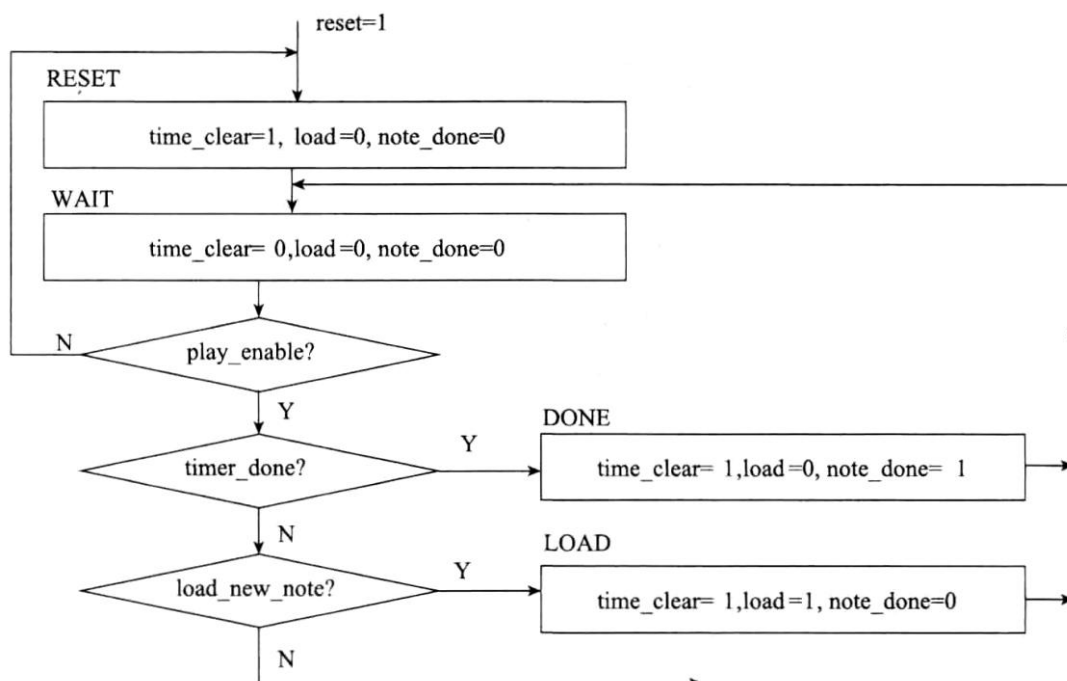


图 11 note_player 控制器的算法流程图

音符定时器为 6 位二进制计数器，beat、timer_clear 分别为使能、清 0 信号，均为高电平有效。定时时间由音长信号 duration_to_load 决定，即 duration_to_load 个 beat 周期，timer_done 为定时结束标志。

子模块 DDS 的功能就是利用 DDS 技术产生正弦样品，其工作原理前面已经介绍了，注意：DDS 模块的输入 k 为 22 位二进制，因此需 FreqROM 输出的 20 位相位增量高位加 2 个 0 后接入 DDS。

6、同步化电路

由于音频编解码接口模块和其他模块采用不同的时钟，因此两者之间的控制及应答信号须进行同步化处理。

本例中音频编解码接口模块的输出信号 NewFrame 的脉冲宽度为一个 audio_clk 时钟周期，需通过同步化处理，产生与 sys_clk 同步且脉冲宽度为一个 sys_clk 时钟周期的信号 ready。电路如图 12 所示，由同步器和脉冲宽度变换电路组成。

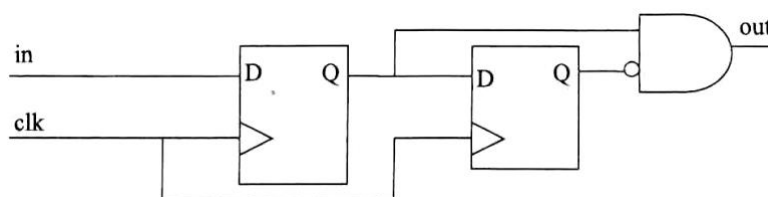


图 12 同步化电路

7、时钟管理模块(DCM)

IP 内核时钟管理模块的输入时钟 clk 频率为 100MHz，产生 100MHz 的系统时钟和 12.50MHz 的音频时钟。

四、实验设备

- (1)装有 Vivado 和 ModelSim SE 软件的计算机。
- (2)Nexys Video Artix-7FPGA 多媒体音视频智能互联开发系统。
- (3)有源音箱或耳机。

五、实验内容

- (1)从网站下载提供的文件。
- (2)按照实验 15 介绍的 DDS 原理，编写 DDS 模块的 Verilog HDL 代码，并用 ModelSim 仿真验证。
- (3)编写 mcu 模块的 Verilog HDL 代码，并用 ModelSim 仿真验证。
- (4)编写 song_reader 模块的 Verilog HDL 代码，并用 ModelSim 仿真验证。
- (5)编写 note_player 模块的 VerilogHDL 代码，并用 ModelSim 仿真验证。
- (6)编写 1000 分频器模块的 Verilog HDL 代码及其测试代码,并用 ModelSim 仿真。
- (7)编写同步化电路的 Verilog HDL 代码及其测试代码，并用 ModelSim 仿真。
- (8)编写次顶层 music_player 模块的 Verilog HDL 代码及其测试代码。
- (9)新建 music_player 的 Vivado 工程,编写顶层 music_player_top 模块的 Verilog HDL 代码，生成符合要求的 DCM 内核，添加 music_player 模块及其子模块、音频编解码接口模块的网表文件 AudioInterface.edf和端口文件 AudioInterface.v、按键处理模块提供综合网表文件 button_press_unit.edf 和端口文件 button_press_unit.v 等。对工程进行综合、约束、实现，并下载工程文件到 NexysVideo 开发板中。
- (10)将耳机接入实验开发板音频输出插座，操作 reset(中间按钮)、play/pause(右边按钮)、next(下面按钮)三个按键，试听耳机中的乐曲并观察实验板上指示灯变化情况，验证设计结果是否正确。

六、实验代码设计与仿真

1、DDS 模块

(1) DDS 模块的代码设计

DDS 模块由顶层设计模块 dds.v 和子模块加法器 full_adder.v、D 触发器 dffre.v 组成。需要说明的是，加法器代码 full_adder.v 采用的是全加器代码，由于本实验的加法器没有进位输入，因此在顶层模块调用时直接将 ci 设为 1'b0。具体的代码附在 solution 文件夹中，此处不做展示。

(2) DDS 模块的 ModelSim 仿真

DDS 模块的 ModelSim 仿真波形如图 13 所示。如图，k 越大，正弦波 sample 的频率越大，且频率的变化点（即波形的变化点）与 k 的变化点一致，都是图中红线时刻，这些特征与该模块需要实现的功能一致，该模块程序编写正确，可以正常运行。

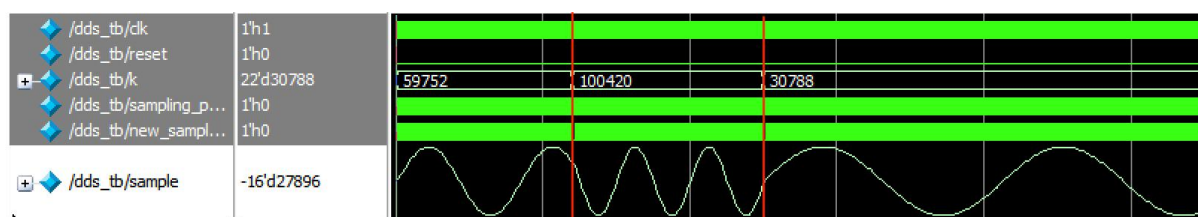


图 13 DDS 模块的 ModelSim 仿真波形

2、mcu 模块

(1) mcu 模块的代码设计

mcu 模块由顶层设计模块 mcu.v 和子模块控制器 mcu_ctrl.v、2 位二进制计数器 counter_n.v 组成。具体的代码附在 solution 文件夹中，此处不做展示。

(2) mcu 模块的 ModelSim 仿真

mcu 模块的 ModelSim 仿真波形如图 14 所示。将第一首歌的波形区域放大，并增加 NextSong 和 state 的波形，如图 15 所示。

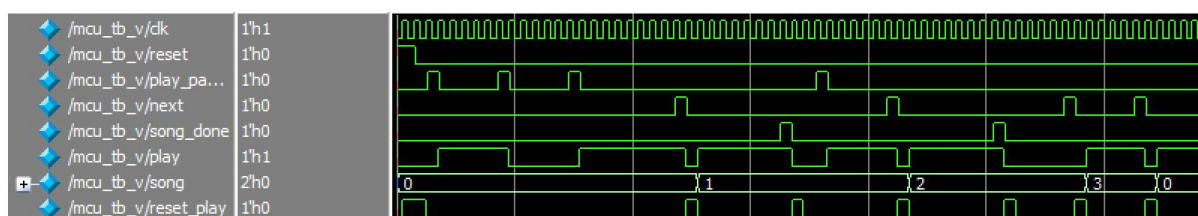


图 14 mcu 模块的 ModelSim 仿真波形

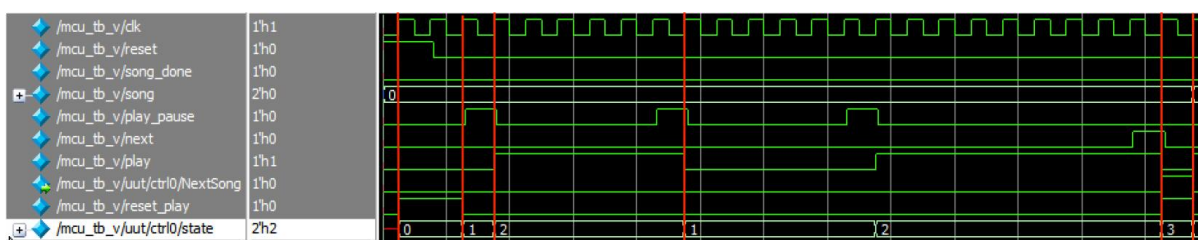


图 15 mcu 模块的 ModelSim 仿真波形局部放大图

由图 15，可以得出如下的分析。

在第一条红线时刻，reset=1，时钟上升沿到来后，控制器状态跳为 0 状态，即 RESET，输出 play=0，NextSong=0，reset_play=1；

在第二条红线时刻，时钟上升沿到来后，控制器状态跳为 1 状态，即 PAUSE，输出 play=0，NextSong=0，reset_play=0；

在第三条红线时刻，play_pause=1，时钟上升沿到来后，控制器状态跳为 2 状态，即 PLAY，输

出 play=1, NextSong=0, reset_play=0;

在第四条红线时刻, play_pause=1, 时钟上升沿到来后, 控制器状态跳为 1 状态, 即 PAUSE, 输出 play=0, NextSong=0, reset_play=0;

在第五条红线时刻, play_pause=0, next=1, 时钟上升沿到来后, 控制器状态跳为 3 状态, 即 NEXT, 输出 play=0, NextSong=1, reset_play=1;

在第六条红线时刻, 时钟上升沿到来后, 控制器状态跳为跳为 2 状态, 即 PLAY, 输出 play=1, NextSong=0, reset_play=0, 进入下一首歌的播放。

同理, 可以得到后三首歌的波形分析。

将这些分析与图 7 的算法流程图比较, 可以发现两者相吻合, 波形特征与该模块需要实现的功能一致, 该模块程序编写正确, 可以正常运行。

3、song_reader 模块

(1) song_reader 模块的代码设计

song_reader 模块由顶层设计模块 song_reader.v 和子模块控制器 song_reader_ctrl.v、地址计数器 counter_n.v、乐曲存放模块 song_rom.v 和结束判断模块 song_over.v 组成。其中结束判断模块采用状态机方法实现。需要说明的是, 我保留了 song_rom 中的第 1 首歌, 而后面的 3 首歌被我依次替换为了黄霄雲的《光之黎明》《盲选》《星辰大海》。具体的代码附在 solution 文件夹中, 此处不做展示。

(2) song_reader 模块的 ModelSim 仿真

song_reader 模块的 ModelSim 仿真波形如图 16 所示。将波形起点, 末尾和中段依次放大, 并增加 state 的波形, 分别如图 17、图 18、图 19 所示。

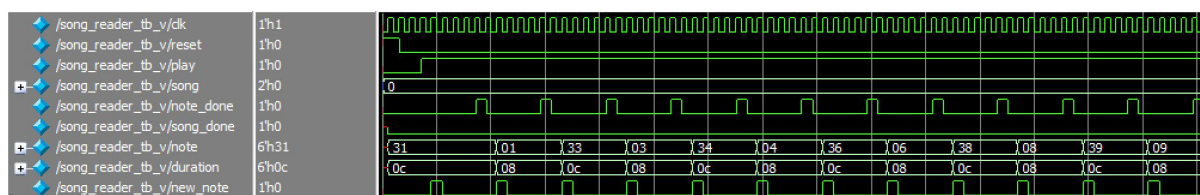


图 16 song_reader 模块的 ModelSim 仿真波形

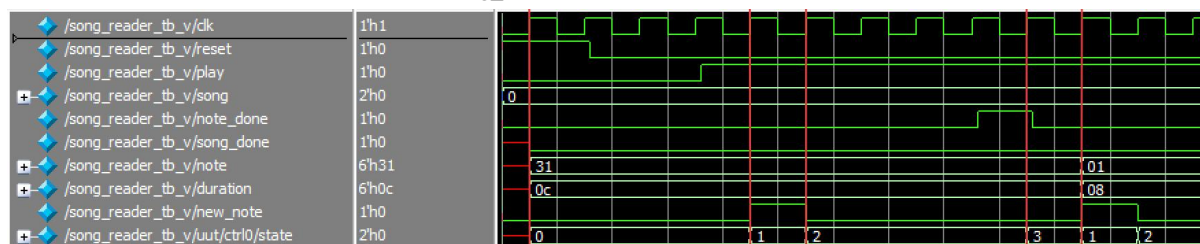


图 17 song_reader 模块的 ModelSim 仿真波形起点放大图

由图 17, 可以得出如下的分析。

在第一条红线时刻, reset=1, 时钟上升沿到来后, 控制器状态跳为 0 状态, 即 RESET, 输出 new_note=0。之后, 由于 play=0, 故控制器状态始终保持为 0 状态 (RESET);

在第二条红线时刻, play=1, 时钟上升沿到来后, 控制器状态跳为 1 状态, 即 NEW_NOTE, 输出 new_note=1;

在第三条红线时刻, 时钟上升沿到来后, 控制器状态自然跳为 2 状态, 即 WAIT, 输出 new_note=0。之后, 由于 play=1, note_done=0, 故控制器状态始终保持为 2 状态 (WAIT);

在第四条红线时刻, play=1, note_done=1, 时钟上升沿到来后, 控制器状态跳为 3 状态, 即 NEXT_NOTE, 输出 new_note=0;

在第五条红线时刻, 时钟上升沿到来后, 控制器状态自然跳为 1 状态, 即 NEW_NOTE, 输出 new_note=1。

将这些分析与图 9 的算法流程图比较，可以发现两者相吻合，波形特征与该模块需要实现的功能一致，控制器程序编写正确，可以正常运行。

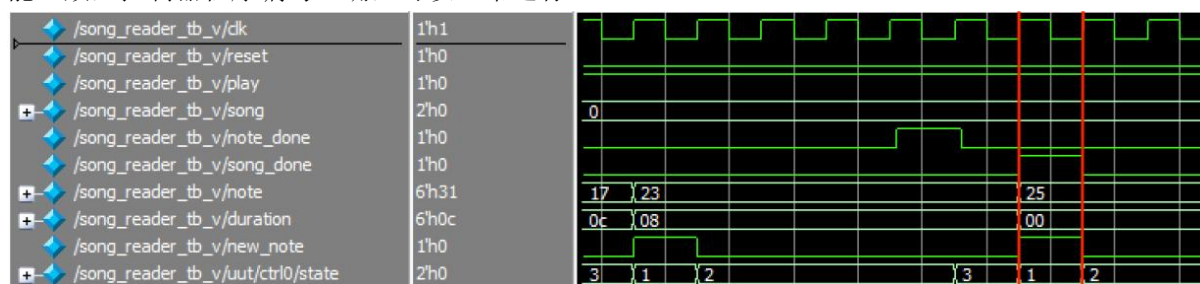


图 18 song_reader 模块的 ModelSim 仿真波形末尾放大图

由图 18，在第一条红线时刻，duration=0，时钟上升沿到来后，输出了一个时钟周期宽度的脉冲信号 song_done=1，与该模块需要实现的功能一致，结束判断模块程序编写正确，可以正常运行。

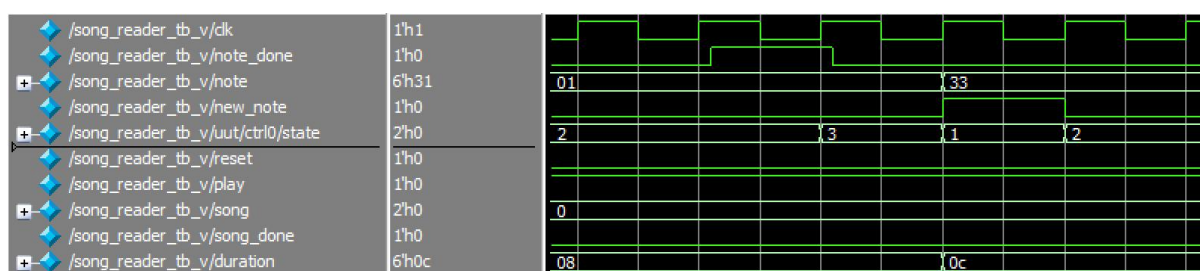


图 19 song_reader 模块的 ModelSim 仿真波形中段放大图

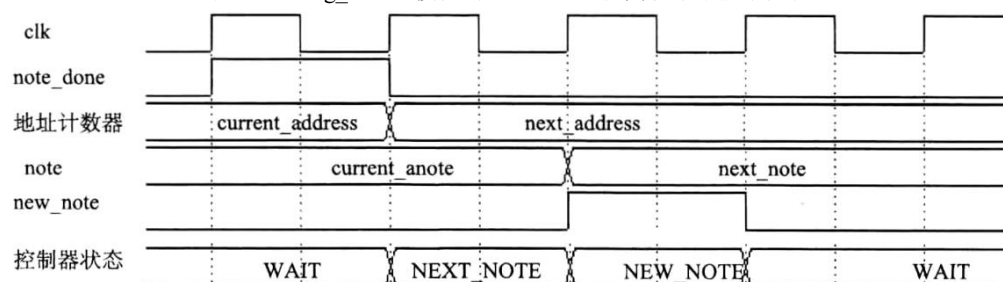


图 20 song_reader 控制器、地址计数器和 ROM 的时序关系

由图 19，将其与图 20 所示的时序关系对比，发现两者吻合，说明该模块功能正常，程序编写正确，可以正常运行。

4、note_player 模块

(1) note_player 模块的代码设计

note_player 模块由顶层设计模块 note_player.v 和子模块控制器 note_player_ctrl.v、音符节拍定时器 note_timer.v、D 触发器 dffre.v、只读存储器 frequency_rom.v、DDS 模块 dds.v 组成。具体的代码附在 solution 文件夹中，此处不做展示。

(2) note_player 模块的 ModelSim 仿真

note_player 模块的 ModelSim 仿真波形如图 21 所示。

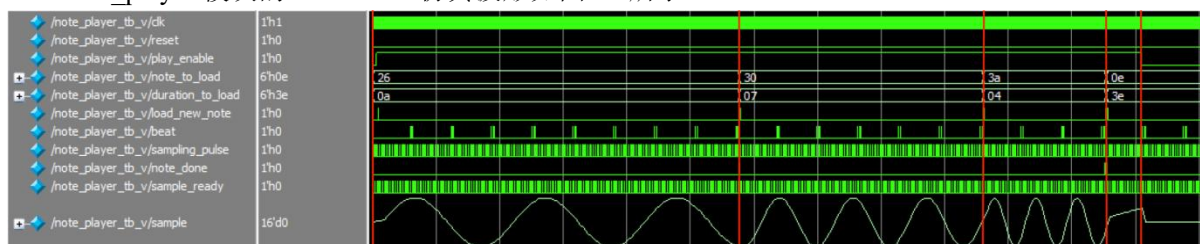


图 21 note_player 模块的 ModelSim 仿真波形

如图 21，note_to_load 越大，正弦波 sample 的频率越大，且频率的变化点（即波形的变化点）与 note_to_load 的变化点一致，都是图中前四条红线时刻。此外，当 play_enable=0 时，sample=0，这些特征与该模块需要实现的功能一致，该模块程序编写正确，可以正常运行。

将图 21 波形前两条红线处依次放大，分别如图 22、图 23 所示。

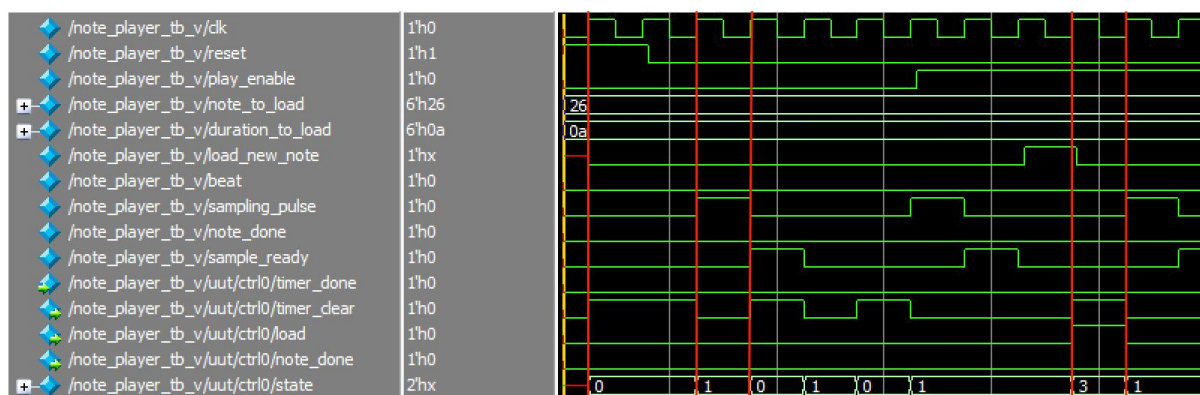


图 22 note_player 模块的 ModelSim 仿真波形第一条红线放大图

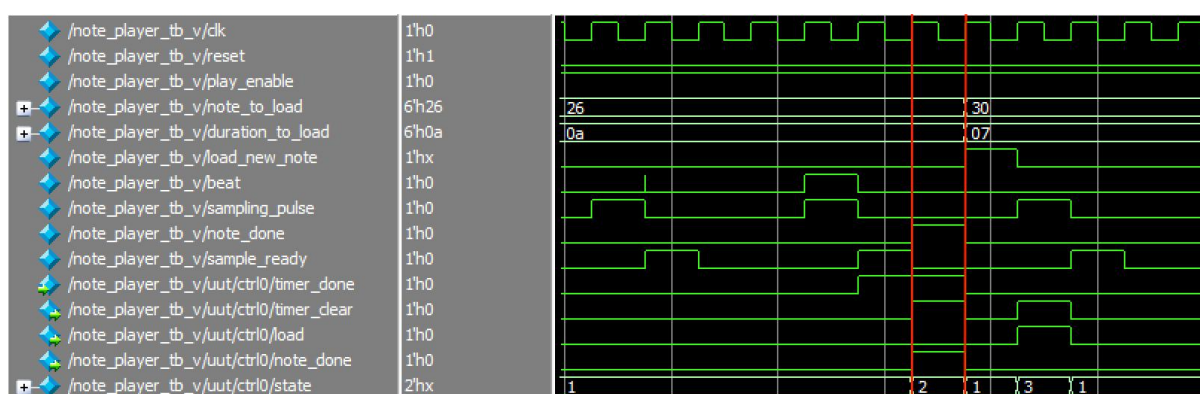


图 23 note_player 模块的 ModelSim 仿真波形第二条红线放大图

由图 22，可以得出如下的分析。

在第一条红线时刻，reset=1，时钟上升沿到来后，控制器状态跳为 0 状态，即 RESET，输出 time_clear=1，load=0，note_done=0；

在第二条红线时刻，时钟上升沿到来后，控制器状态自然跳为 1 状态，即 WAIT，输出 time_clear=0，load=0，note_done=0；

在第三条红线时刻，play_enable=0，时钟上升沿到来后，控制器状态跳为 0 状态，即 RESET，输出 time_clear=1，load=0，note_done=0；

在第四条红线时刻，play_enable=1，timer_done=0，load_new_note=1，时钟上升沿到来后，控制器状态跳为 3 状态，即 LOAD，输出 time_clear=1，load=1，note_done=0；

在第五条红线时刻，时钟上升沿到来后，控制器状态自然跳为 1 状态，即 WAIT，输出 time_clear=0，load=0，note_done=0；

由图 23，可以得出如下的分析。

在第一条红线时刻，play_enable=1，timer_done=1，时钟上升沿到来后，控制器状态跳为 2 状态，即 DONE，输出 time_clear=1，load=0，note_done=1；

在第二条红线时刻，时钟上升沿到来后，控制器状态自然跳为 1 状态，即 WAIT，输出 time_clear=0，load=0，note_done=0；

将这些分析与图 11 的算法流程图比较，可以发现两者相吻合，波形特征与该模块需要实现的功能一致，控制器程序编写正确，可以正常运行。

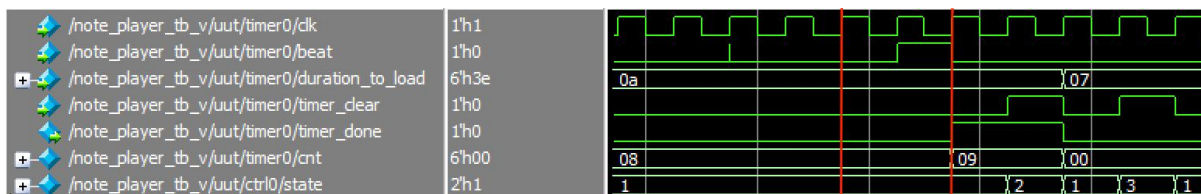


图 23 note_player 模块的音符节拍定时器的 ModelSim 仿真波形

将图 21 波形增加音符节拍定时器的波形并放大,如图 24 所示。如图,在第一条红线时刻,beat=0,使能信号为低电平,定时器不计数,cnt 保持为 8;在第二条红线时刻,beat=1,定时器计数,由于此时 duration=10,cnt=9,定时结束,timer_done=1,时钟上升沿到来后,控制器状态由 1 状态(WAIT)跳为 2 状态(DONE),波形特征与该模块需要实现的功能一致,音符节拍定时器程序编写正确,可以正常运行。

5、1000 分频器模块

(1) 1000 分频器模块的代码设计

1000 分频器模块是模 1000 的计数器,其状态位数为 10 位,因此,只需在 counter_n.v 中令分频比 n=1000,状态位数 counter_bits=10 即可得到 1000 分频器,其测试代码为 counter_n_tb.v。具体的代码附在 solution 文件夹中,此处不做展示。

(2) 1000 分频器模块的 ModelSim 仿真

1000 分频器模块的 ModelSim 仿真波形如图 24 所示。如图,en=1 时,时钟上升沿到来后,分频器正常计数。当 q=999,en=1 时,在图中红线时刻,时钟上升沿到来后,进位输出 co=1,且计数值 q 回到 0,这些特征与该模块需要实现的功能一致,该模块程序编写正确,可以正常运行。

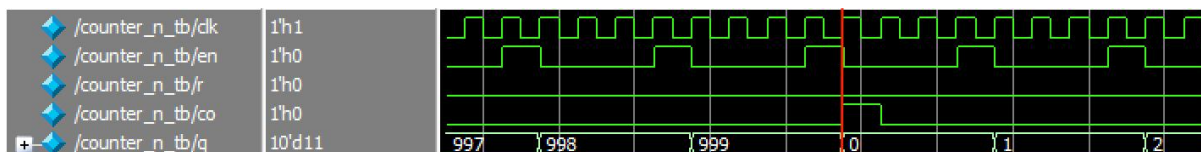


图 24 1000 分频器模块的 ModelSim 仿真波形

6、同步化电路

(1) 同步化电路的代码设计

如图 12 所示,同步化电路由两个 D 触发器和一个与门构成,按此原理,可编写出同步化电路的代码 syn.v,其测试代码为 syn_n_tb.v。具体的代码附在 solution 文件夹中,此处不做展示。

(2) 同步化电路的 ModelSim 仿真

同步化电路的 ModelSim 仿真波形如图 25 所示。如图,一段时间后,输出信号 out 与输入信号 in 实现同步,且 out 的脉冲宽度为一个时钟周期,这些特征与该模块需要实现的功能一致,该模块程序编写正确,可以正常运行。

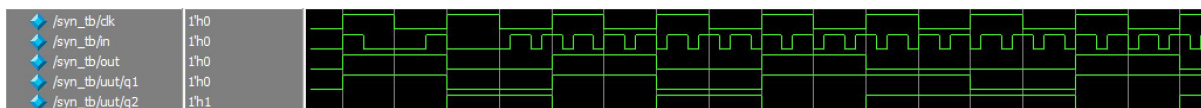


图 25 同步化电路的 ModelSim 仿真波形

7、次顶层 music_player 模块

(1) 次顶层 music_player 模块的代码设计

次顶层 music_player 模块由顶层设计模块 music_player.v 和子模块主控制器 mcu.v、乐曲读取模块 song_reader.v、音符播放模块 note_player.v、同步化电路 syn.v 和节拍基准产生器 counter_n.v 组成,其中所有的子模块都已经在前面分析并编写了。具体的代码附在 solution 文件夹中,此处不做展示。

(2) 次顶层 music_player 模块的 ModelSim 仿真

次顶层 music_player 模块的 ModelSim 仿真波形如图 26 所示。如图，该波形特征与该模块需要实现的功能一致，该模块程序编写正确，可以正常运行。

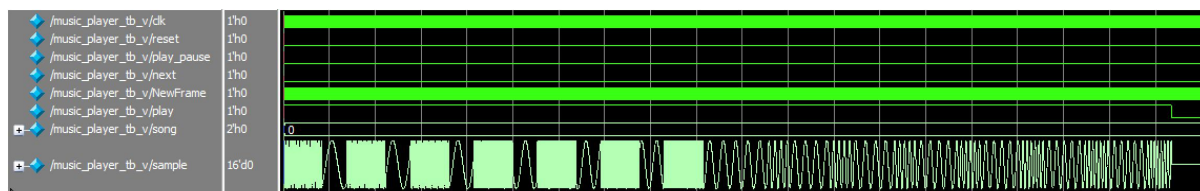


图 26 次顶层 music_player 模块的 ModelSim 仿真波形

8、建立 Vivado 工程

在 Vivado 软件中，生成符合要求的 DCM 内核，将所有程序导入，对工程进行综合，得到的原理图如图 27 所示。将该原理图与图 5 的原理框图进行对照，可以看出该原理图符合设计需求。

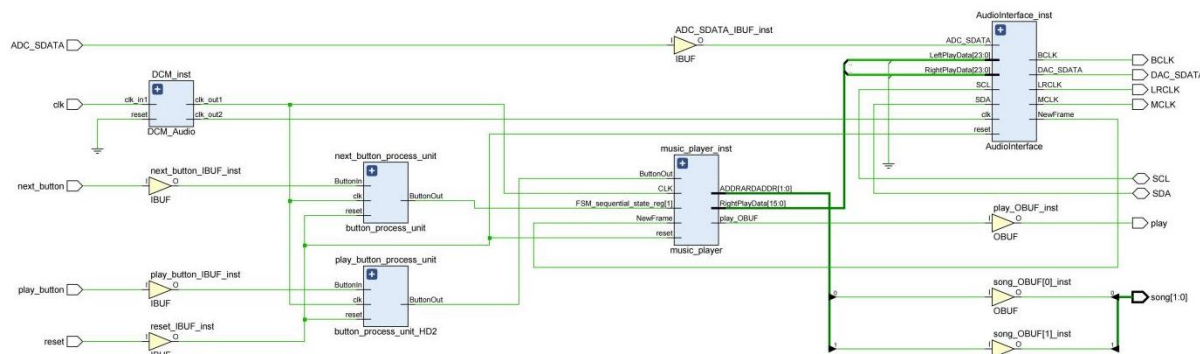


图 27 Vivado 原理图

七、开发板测试

为检验本次实验任务是否能够在开发板上实现，将其工程文件上传到 Nexys Video 开发板上进行测试。经测试，开发板能够正常播放歌曲，LED 灯能够正常亮灭，play/pause button、next button、reset 三个按键能够正常使用，各功能都能正常实现，本次实验成功。

八、总结、心得

通过本次音乐播放器实验，我对数字系统设计中的“自顶而下”设计方法有了更深入的理解。从顶层模块的划分到底层子模块的实现，整个设计过程逻辑清晰、层次分明，尤其是主控制器、乐曲读取模块和音符播放模块之间的协同工作，让我体会到了模块化设计的优势。

在 DDS 模块的设计与仿真中，我掌握了如何通过相位累加器和正弦查找表生成不同频率的正弦波，进一步理解了频率控制字与输出频率之间的关系。通过 ModelSim 波形仿真，我能够直观地观察到频率变化对输出波形的影响，这加深了我对 DDS 技术原理的理解。

在 mcu 模块和 song_reader 模块的设计中，我学习了如何通过状态机实现按键响应、乐曲切换和播放控制。仿真波形的分析与算法流程图的对比，使我更加注重设计规范与时序逻辑的正确性。尤其是在处理 next 按键和 play/pause 按键时，我意识到消抖动和同步化处理对系统稳定性的重要性。

在 note_player 模块的设计中，我掌握了如何将音符标记映射为相位增量，并控制音符的播放时长。通过与 song_reader 模块的交互，我进一步理解了模块间的接口设计与握手协议的重要性。

此外，Vivado 工程的综合、约束与开发板测试环节，让我体验了从代码设计到硬件验证的完整流程。在我第一次上板测试时，有两个按键无效。这使我仔细检查了每个程序，在反复试验后终于找出了问题所在。当耳机中成功播放出乐曲，LED 灯正确指示播放状态和曲目序号时，整个系统的成功运行让我感到非常有成就感。

总的来说，本次实验不仅巩固了我在数字系统设计、DDS 技术、状态机设计、仿真验证等方面的知识，也提升了我的工程实践能力和问题分析能力。在今后的学习中，我将继续保持这种从原理到实现、从仿真到实测的严谨态度，不断提升自己的综合设计能力。

九、思考题

(1)在实验中，为什么 next_button、play_pause_button 两个按键需要消颤动及同步化处理，而 reset 按键不需要消颤动及同步化处理？

reset 是异步置零信号。当 reset=1 时，系统强制置零，之后无论 reset 是否因抖动产生短暂跳变，系统内部已被置零的状态不会再次改变，因此抖动不会影响系统功能。

而 next_button 和 play_pause_button 控制系统的状态转换为切换歌曲、播放/暂停等。若这些按键存在抖动，可能导致一次按下被识别为多次触发，例如 next 抖动会使歌曲连续跳转多首，play_pause 抖动会使系统在播放与暂停之间反复切换，造成输出断续、功耗增加、用户体验下降，因此必须进行消抖动和同步化处理。

(2)在主控制器(mcu)设计中，是否存在接收不到按键信息？若存在，概率多大？有没必要修改设计？

存在接收不到按键信息的情况。根据控制器的 ASM 图，在 RESET→PAUSE 转换以及 NEXT→PLAY 转换的过程中，并未检测 play_pause 和 next 按键，此时按下这两个按键会被忽略。由于这些状态转换仅持续一个时钟周期，而按键按下持续时间远长于该周期，因此接收不到的概率极低，没有必要修改设计。